SIT NAGPUR • Dept. of Computer Science • JavaScript Lab Tanmay Sankulwar | PRN: 24070521058

EXPERIMENT 11

Student Name: Tanmay Sankulwar
PRN: 24070521058
File Path: `EXP11/index.html`, `EXP11/script.js`, `EXP11/scheduler.js`, `EXP11/style.css`

Experiment Title

Class Countdown Scheduler using ES6 Modules and Asynchronous Timing (`setInterval` & `clearInterval`)

Objective

1. To understand and apply modern ES6 JavaScript Module architecture (`import` and `export`) to segregate core computational logic from DOM presentation layers.
2. To work with the JavaScript `Date` API (`new Date()`, `.getTime()`, `.toLocaleString()`, `.toISOString()`, and `.getTimezoneOffset()`) for accurate timestamp calculations.
3. To implement asynchronous interval execution using `setInterval()` and prevent memory leaks or redundant timers using `clearInterval()`.
4. To perform millisecond difference decomposition into days, hours, minutes, and seconds using integer arithmetic and modulo operations.
5. To implement uniform two-digit numerical formatting using `String().padStart(2, "0")`.
6. To enforce client-side form validation ensuring target lecture timestamps are set strictly to valid future moments.

Software / Tools Required

1. **Operating System:** Windows 10 / 11
2. **Development Environment:** Visual Studio Code / Antigravity IDE
3. **Web Browser:** Google Chrome / Modern Web Browser
4. **Languages & Technologies:** HTML5, Modern CSS3 (CSS Grid, Responsive Flexbox, Shadow Cards), Modern JavaScript (ES6 Modules)

Program / Task

Task 11 — Dynamic Class Countdown Scheduler

Files: `EXP11/index.html`, `EXP11/scheduler.js`, `EXP11/script.js`, `EXP11/style.css`

The application provides a responsive class schedule planner and live countdown timer built with a modular JavaScript architecture: - **Modular Division of Concerns:** - `scheduler.js` serves as an independent ES6 module exporting `getTimeRemaining(targetDate)` and `formatDate(date)`. - `script.js` imports these utilities and acts as the controller managing DOM state, event listeners, and interval timers. - **Dynamic Input Validation:** - On page load, the controller normalizes the local timezone offset to restrict the `<input type="datetime-local">` minimum boundary (`min`), disallowing past dates from being selected. - **Live 1-Second Interval Engine:** - Upon form submission (`submit`), existing interval timers are safely cleared (`clearInterval(countdownInterval)`), and a fresh 1000ms loop is initiated with `setInterval(updateCountdown, 1000)`. - **4-Digit Countdown Grid:** - Displays remaining Days, Hours, Minutes, and Seconds inside dedicated styled metric cards. - **Expiration Trigger:** - Once the target moment is reached (`difference <= 0`), the timer halts cleanly, zeros all cards, and updates the status banner to `"Your class time has arrived!"`.

Source Code

Main Application Markup (`EXP11/index.html`)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Class Countdown Scheduler</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <header>
            <h1>Class Countdown</h1>
            <p>Stay organized. Be on time.</p>
        </header>

        <section class="timer-card">
            <h2 id="eventTitle">Next Class</h2>
            <p id="eventDate">Select your class date and time</p>

            <div class="countdown">
                <div class="time-box">
                    <span id="days">00</span>
                    <small>Days</small>
                </div>

                <div class="time-box">
                    <span id="hours">00</span>
                    <small>Hours</small>
                </div>

                <div class="time-box">
                    <span id="minutes">00</span>
                    <small>Minutes</small>
                </div>

                <div class="time-box">
                    <span id="seconds">00</span>
                    <small>Seconds</small>
                </div>
            </div>

            <p id="status">Set your next class below.</p>
        </section>

        <section class="form-card">
            <h2>Schedule a Class</h2>

            <form id="scheduleForm">
                <label for="className">Class Name</label>
                <input type="text" id="className"
                       placeholder="e.g. JavaScript Lecture" required>

                <label for="classDate">Date and Time</label>
                <input type="datetime-local" id="classDate" required>

                <button type="submit">Start Countdown</button>
            </form>
        </section>
```

```
        <footer>
            <p>Dynamic Countdown Scheduler | JavaScript Practical 11</p>
        </footer>
    </div>

    <script type="module" src="script.js"></script>
</body>
</html>
```

ES6 Logic Module (`EXP11/scheduler.js`)

```
export function getTimeRemaining(targetDate) {
    const currentTime = new Date().getTime();
    const targetTime = new Date(targetDate).getTime();
    const difference = targetTime - currentTime;

    if (difference <= 0) {
        return {
            days: 0,
            hours: 0,
            minutes: 0,
            seconds: 0,
            expired: true
        };
    }

    return {
        days: Math.floor(difference / (1000 * 60 * 60 * 24)),
        hours: Math.floor((difference / (1000 * 60 * 60)) % 24),
        minutes: Math.floor((difference / (1000 * 60)) % 60),
        seconds: Math.floor((difference / 1000) % 60),
        expired: false
    };
}

export function formatDate(date) {
    return new Date(date).toLocaleString();
}
```

DOM Controller Script (`EXP11/script.js`)

```
import { getTimeRemaining, formatDate } from "./scheduler.js";

const form = document.getElementById("scheduleForm");
const className = document.getElementById("className");
const classDate = document.getElementById("classDate");

const eventTitle = document.getElementById("eventTitle");
const eventDate = document.getElementById("eventDate");
const status = document.getElementById("status");

let countdownInterval = null;
let targetDate = null;

function updateCountdown() {
    if (!targetDate) {
        return;
    }

    const time = getTimeRemaining(targetDate);

    document.getElementById("days").textContent =
        String(time.days).padStart(2, "0");

    document.getElementById("hours").textContent =
        String(time.hours).padStart(2, "0");

    document.getElementById("minutes").textContent =
        String(time.minutes).padStart(2, "0");

    document.getElementById("seconds").textContent =
        String(time.seconds).padStart(2, "0");

    if (time.expired) {
        status.textContent = "Your class time has arrived!";
        clearInterval(countdownInterval);
        countdownInterval = null;
    } else {
        status.textContent = "Countdown is running...";
    }
}

form.addEventListener("submit", function (event) {
    event.preventDefault();

    const selectedName = className.value.trim();
    const selectedDate = new Date(classDate.value);

    if (!selectedName || !classDate.value ||
        Number.isNaN(selectedDate.getTime()) ||
        selectedDate.getTime() <= Date.now()) {
        status.textContent = "Please enter a valid future date and time.";
        return;
    }

    targetDate = selectedDate;

    eventTitle.textContent = selectedName;
    eventDate.textContent = formatDate(targetDate);

    if (countdownInterval !== null) {
        clearInterval(countdownInterval);
    }
```

```
    updateCountdown();
    countdownInterval = setInterval(updateCountdown, 1000);
});

const now = new Date();
now.setMinutes(now.getMinutes() - now.getTimezoneOffset());

classDate.min = now.toISOString().slice(0, 16);
```

Explanation / Concepts

1. **ES6 Module Architecture (`type="module"`):**
2. Declaring `<script type="module" src="script.js">` enables native browser ES module support, executing in strict mode (`use strict`) with deferred evaluation.
3. Using `export function` in `scheduler.js` and `import { ... }` in `script.js` encapsulates helper logic into reusable units, preventing global namespace pollution.
4. **Temporal Mathematics & Difference Decomposition:**
5. Epoch milliseconds: `const difference = targetTime - currentTime;`
6. Days: `Math.floor(difference / (1000 * 60 * 60 * 24))`
7. Hours: `Math.floor((difference / (1000 * 60 * 60)) % 24)`
8. Minutes: `Math.floor((difference / (1000 * 60)) % 60)`
9. Seconds: `Math.floor((difference / 1000) % 60)`
10. **Managing Asynchronous Intervals:**
11. `setInterval(callback, interval)` executes repeatedly every given interval.
12. If a new class schedule is submitted while a countdown is already running, calling `clearInterval(countdownInterval)` prevents concurrent timer collisions and memory leaks.
13. **Dynamic Two-Digit Zero-Padding (`padStart`):**
14. `String(num).padStart(2, "0")` guarantees that single-digit metrics (e.g. `9`) render with leading zeros (`"09"`), preserving alignment across countdown boxes.
15. **Local Timezone Offset Synchronization:**
16. HTML `<input type="datetime-local">` requires an ISO string matching local clock time. Subtracting `now.getTimezoneOffset()` aligns the `toISOString()` output with the user's regional timezone for the `min` constraint.

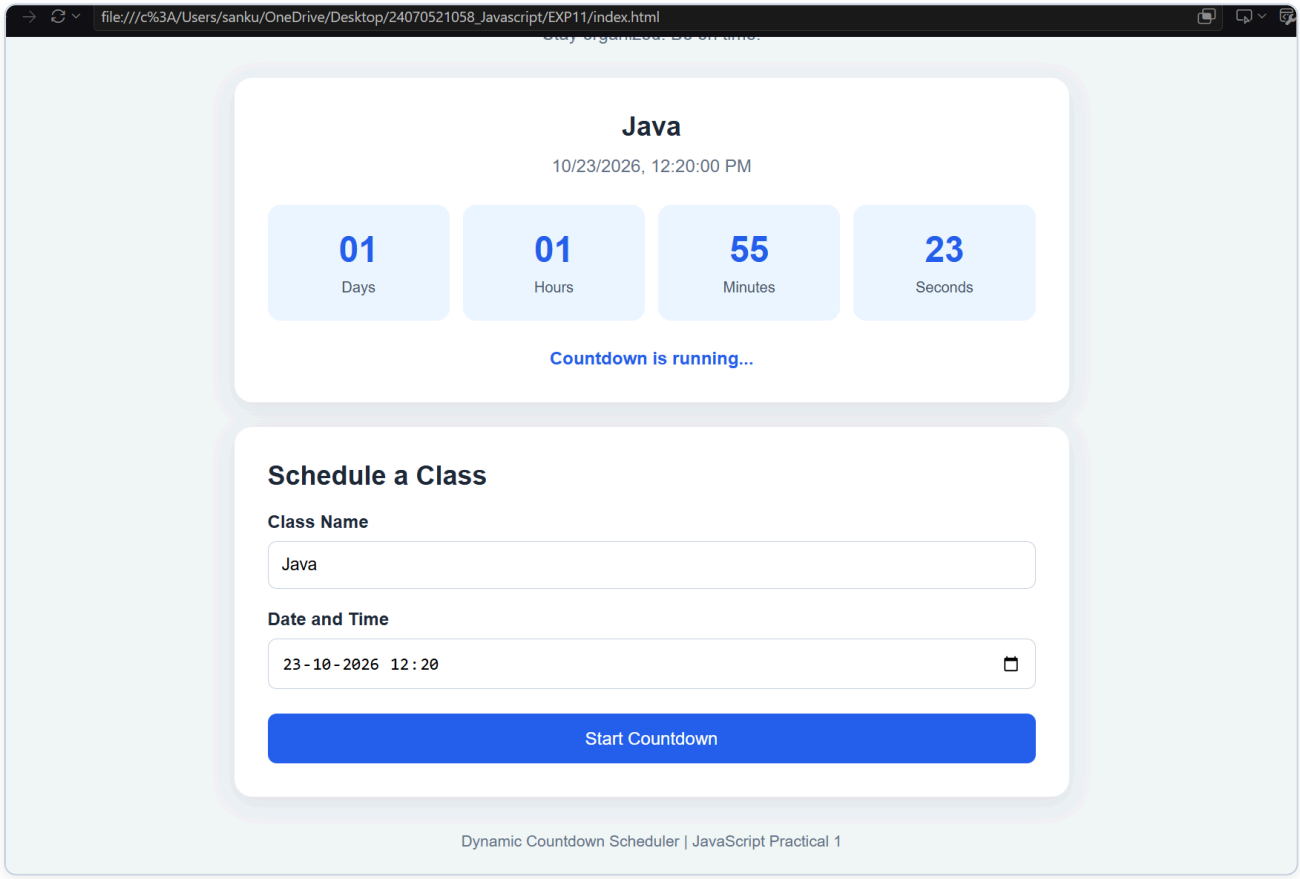
Output

1. **Initial Idle State:** Displays the countdown cards initialized to `00 Days : 00 Hours : 00 Minutes : 00 Seconds` with helper text `"Select your class date and time"`.

2. **Active Countdown:** Upon selecting a valid class name (e.g. `"JavaScript Practical Examination"`) and a future date/time, the title updates, the formatted timestamp is shown via `.toLocaleString()`, and all 4 metric boxes begin counting down live each second.
3. **Arrival Notification:** When the clock hits zero, `clearInterval()` halts execution, cards freeze at zero, and the banner alerts `"Your class time has arrived!"`.

Screenshot

Class Countdown Scheduler Output



Screenshot 2026-10-22 102441.png

Result / Conclusion

Experiment 11 was successfully implemented and verified. The practical demonstrated the utility of modern modular JavaScript: separating date parsing and math functions into an independent ES6 module, maintaining accurate second-by-second countdown state using `setInterval()` and `clearInterval()`, and applying user-friendly defensive datetime validation.